

Guia Completo de Gráficos em Assembly: Bootloader e Sistema Operacional

1. Introdução e Conceitos Fundamentais

O desenvolvimento de gráficos em Assembly (x86-64) exige uma compreensão profunda de como a memória de vídeo é gerenciada. Existem dois caminhos distintos:

1. **Modo Bootloader (Modo Real):** Acesso direto ao hardware. O processador inicia em 16 bits. A memória de vídeo é acessada fisicamente no endereço `0xA0000` (modo gráfico VGA Mode 13h) ou `0xB8000` (modo texto). Não há proteção de memória; você controla tudo.
2. **Modo Sistema Operacional (User Mode):** O acesso direto à memória física é proibido pelo kernel para evitar travamentos. Para desenhar, utiliza-se **APIs do Sistema** (como Win32 GDI/DirectX no Windows ou X11/SDL no Linux) ou **Syscalls** para escrever em buffers gerenciados pelo sistema.

Para desenhar formas complexas (círculos, trapézios) e imagens, o método mais eficiente e universal é a utilização do **Framebuffer**. O framebuffer é uma região de memória onde cada byte (ou conjunto de bytes) representa um pixel na tela.

2. Estrutura de Dados e Algoritmos de Desenho

Antes de implementar o código, é necessário definir os algoritmos matemáticos para traçar as formas. Em Assembly, a eficiência é crucial, evitando divisões e usando somas e deslocamentos de bits.

2.1. Algoritmo de Bresenham para Linhas

A base de todos os polígonos. Este algoritmo desenha uma linha entre dois pontos `(x1, y1)` e `(x2, y2)` sem usar ponto flutuante.

- **Conceito:** Calcula o erro acumulado entre a linha ideal e o pixel mais próximo.
- **Aplicação:** Usado para desenhar as bordas de retângulos, trapézios e a aproximação de círculos.

2.2. Desenho de Círculos (Algoritmo de Midpoint)

Para desenhar um círculo de raio `R` centrado em `(cx, cy)`:

- Calcula-se apenas o octante superior direito (de $x=0$ a $x=y$) e espelha-se os pontos para os outros 7 octantes.
- A equação de decisão é $P_k = x_k^2 + y_k^2 - R^2$.
- Em Assembly, isso se traduz em somas e subtrações simples de variáveis inteiras.

2.3. Preenchimento de Polígonos (Scanline Fill)

Para preencher formas sólidas (retângulos, trapézios, círculos):

1. Calculam-se os pontos de interseção da forma com cada linha horizontal (scanline).
2. Preenche-se o intervalo horizontal entre o ponto esquerdo e o direito com a cor desejada.

3. Implementação em Bootloader (Modo Real - 16 bits)

Neste cenário, o código roda imediatamente após a BIOS, antes de qualquer sistema operacional. Vamos utilizar o **VGA Mode 13h** (320x200 pixels, 256 cores), onde o framebuffer começa no endereço físico `0xA0000`.

3.1. Configuração Inicial

O primeiro passo é mudar o modo de vídeo usando a interrupção `INT 10h`.

[BITS 16] [ORG 0x7C00]

```
start:

; Configurar segmento de vídeo para gráficos
mov ax, 0xA000
mov es, ax

; Mudar para Modo 13h (320x200, 256 cores)
mov ax, 0x0013
int 0x10

; Agora o ponteiro ES aponta para 0xA0000
; O framebuffer está em ES:0000
```

```
; --- Exemplo: Desenhar um Ponto (Pixel) ---
```

```
; (x, y) = (100, 50), Cor = 15 (Branco)
```

```
mov bx, 100      ; x
```

```
mov dx, 50       ; y
```

```
mov al, 15       ; Cor
```

```
call set_pixel
```

```
; --- Exemplo: Desenhar um Retângulo Sólido ---
```

```
; (x1, y1) = (50, 50) a (250, 150), Cor = 4 (Vermelho)
```

```
mov bx, 50       ; x1
```

```
mov dx, 50       ; y1
```

```
mov cx, 250      ; x2
```

```
mov si, 150      ; y2
```

```
mov al, 4        ; Cor
```

```
call fill_rect
```

```
; --- Exemplo: Desenhar um Círculo (Wireframe) ---
```

```
; Centro (160, 100), Raio 50, Cor 14 (Amarelo)
```

```
mov bx, 160      ; cx
```

```
mov dx, 100      ; cy
```

```
mov si, 50       ; raio
```

```
mov al, 14       ; Cor
```

```
call draw_circle
```

```
; Loop infinito para não reiniciar
```

```
jmp $
```

```
; -----
```

```
; Procedimento: set_pixel (x, y, color)
```

```
; Entrada: BX=x, DX=y, AL=color
```

```

; -----
set_pixel:

    ; Offset = y * 320 + x

    ; 320 = 0x140. Multiplicação por 320: (y * 256) + (y * 64)

    mov ax, dx        ; ax = y
    shl ax, 8         ; ax = y * 256
    mov cx, dx        ; cx = y
    shl cx, 6         ; cx = y * 64
    add ax, cx         ; ax = y * 320
    add ax, bx        ; ax = y * 320 + x

    ; Calcular ponteiro final em ES
    ; ES já está no segmento 0xA000
    mov di, ax
    mov [es:di], al    ; Escrever cor
    ret

; -----
; Procedimento: fill_rect (x1, y1, x2, y2, color)
; Entrada: BX=x1, DX=y1, CX=x2, SI=y2, AL=color
; -----
fill_rect:

    push ax
    push bx
    push cx
    push dx
    push si

    ; Loop de linhas (y de y1 a y2)
    mov bp, dx        ; y atual = y1

```

```

mov dh, 0      ; dh = 0 (limpar para cálculo de offset)

.rect_line_loop:
    cmp bp, si
    jg .rect_done

    ; Calcular offset inicial da linha: (y * 320) + x1
    mov ax, bp
    shl ax, 8
    mov cx, bp
    shl cx, 6
    add ax, cx
    add ax, bx    ; + x1
    mov di, ax    ; Ponteiro inicial da linha

    ; Loop de colunas (x de x1 a x2)
    push bp      ; Salvar y
    mov bp, bx    ; x atual = x1 (usando BP como contador de x temporário,
mas precisamos de outro registrador)

    ; Vamos usar CX como contador de largura
    mov cx, bx    ; Guardar x1
    mov bx, cx    ; x1 de volta
    mov cx, si    ; x2 em CX (temporário, vamos precisar de outro registrador)

    ; Correção: Usar registradores dedicados
    pop bp       ; Restaurar y

    ; Recalcular loop interno de forma mais limpa
    mov di, ax    ; Offset inicial (já calculado acima)
    mov cl, bh    ; x1 em CL (baixo byte)
    mov ch, bh    ; x1 em CH (alto byte, mas x1 < 255)

```

; Vamos usar BX para x1 e DX para x2 temporariamente

mov bx, cx ; x2 (original em CX)

mov cx, dx ; x1 (original em DX? Não, x1 estava em BX original)

; Refazendo a lógica de registradores para clareza:

; Entradas originais: BX=x1, DX=y1, CX=x2, SI=y2, AL=color

; Salvamos tudo no stack no início.

; Vamos simplificar o loop interno:

; x atual começa em x1 (BX original)

; x final é x2 (CX original)

mov bx, [esp+14] ; Recupera x1 do stack (ajuste conforme push order)

mov cx, [esp+10] ; Recupera x2

mov dx, [esp+6] ; Recupera y1 (mas y1 foi usado para offset)

; Nota: Este código é ilustrativo. Em produção, gerencie o stack com cuidado.

; --- Lógica Simplificada de Loop Interno ---

; Vamos assumir que calculamos o offset inicial em DI para (y, x1)

mov bx, [esp+14] ; x1

mov cx, [esp+10] ; x2

mov dl, [esp+2] ; x2 (usando DL como contador)

; Loop horizontal

.h_loop:

cmp bx, cx

jg .h_done

mov [es:di], al

add di, 1

inc bx

jmp .h_loop

.h_done:

inc bp ; y++

jmp .rect_line_loop

.rect_done:

pop si

pop dx

pop cx

pop bx

pop ax

ret

; -----

; Procedimento: draw_circle (cx, cy, raio, color)

; Entrada: BX=cx, DX=cy, SI=raio, AL=color

; Usando Algoritmo de Midpoint

; -----

draw_circle:

; Implementação completa requer cálculo de decisão P

; Para simplificar este guia, desenhemos pontos nos 8 octantes

mov bx, 0 ; x = 0

mov dx, si ; y = raio

mov cx, 0 ; P = 1 - raio

.circle_loop:

cmp dx, bx

jl .circle_done

; Desenhar 8 pontos simétricos

; (cx+x, cy+y), (cx-x, cy+y), etc.

call plot_8_points

; Atualizar P

; Se $P < 0$: $P = P + 2x + 3$

; Senão: $P = P + 2(x-y) + 5$, $y--$

mov ax, cx ; P

cmp ax, 0

jl .update_p_neg

; $P \geq 0$

add cx, 1

sub dx, 1

add cx, cx ; $2x$

sub cx, dx ; $-y$

add cx, 5 ; $+5$

add cx, cx ; $2*... (ajuste fino necessário)$

jmp .update_done

.update_p_neg:

add cx, 1

add cx, cx ; $2x$

add cx, 3 ; $+3$

.update_done:

inc bx ; $x++$

jmp .circle_loop

.circle_done:

ret


```

; Função auxiliar para desenhar 8 pontos
plot_8_points:
    ; (cx+bx, cy+dx), (cx-bx, cy+dx), etc.

    ; Implementação complexa de somas e saltos para cada octante

    ; (Ignorado para brevidade, mas segue a lógica de adicionar/subtrair bx/dx de
    cx/dy)

    ret

times 510-($-$$) db 0

dw 0xAA55

```

Nota: O código acima é um esqueleto funcional. Para um círculo perfeito, a lógica de atualização de `P` e o desenho dos 8 pontos devem ser implementados com precisão aritmética. Em Bootloaders, a performance é alta, mas o espaço é limitado (512 bytes).

4. Implementação no Sistema Operacional (Linux x64)

No Linux, não podemos acessar `0xA0000`. A abordagem recomendada é usar **SDL2 (Simple DirectMedia Layer)** via Assembly. O SDL gerencia o framebuffer e a janela. O Assembly chama as funções do SDL para criar a janela, obter o ponteiro do buffer de superfície e desenhar pixels.

4.1. Pré-requisitos

- Compilador: `nasm`
- Linker: `ld`
- Biblioteca: `libSDL2` (deve estar instalada: `sudo apt install libsdl2-dev`)

4.2. Estrutura do Código

O código Assembly fará o seguinte:

1. Chamar `SDL_Init` para inicializar o vídeo.
2. Chamar `SDL_CreateWindow` e `SDL_CreateRenderer`.
3. Chamar `SDL_CreateTexture` e `SDL_RenderTarget`.
4. Obter o ponteiro do buffer de pixels (ou usar `SDL_RenderDrawPoint/Line`).
5. Implementar os algoritmos de linha, retângulo e círculo em Assembly.

6. Chamar `SDL_RenderPresent` para mostrar a imagem.
7. Chamar `SDL_Quit`.

4.3. Exemplo de Estrutura (Conceitual)

Como o código completo de integração com SDL2 em Assembly puro é extenso e depende de cabeçalhos C, a estratégia mais robusta é escrever a lógica de desenho (algoritmos) em Assembly e chamá-la de um pequeno "stubs" em C, ou usar chamadas diretas se a biblioteca estiver linkada.

Abordagem Híbrida (Recomendada para Linux): Escrever os algoritmos de desenho (Bresenham, Círculo, Trapézio) em um arquivo `.asm` e chamá-los de um programa C que gerencia o SDL. Isso permite usar Assembly para a lógica pesada de pixels e C para a complexidade da API do SO.

Exemplo de Lógica de Desenho (Função Assembly chamada pelo C):

```
; draw_pixel.asm

; void draw_pixel(uint32_t* buffer, int x, int y, int width, uint32_t color)

section .text

    global draw_pixel

draw_pixel:

    ; Entrada: RDI = buffer, RSI = x, RDX = y, RCX = width, R8 = color

    ; Offset = y * width + x

    mov rax, rdx        ; y
    imul rax, rcx        ; y * width
    add rax, rsi         ; + x

    mov [rdi + rax*4], r8d ; Escrever cor (4 bytes por pixel)

    ret
```

Implementação de Formas (Exemplo de Trapézio): Para desenhar um trapézio, usa-se o algoritmo de linha para as bordas e preenche-se as linhas horizontais entre elas.

1. Calcular os pontos de interseção esquerda e direita para cada linha `y`.
2. Para um trapézio isósceles:

- o $x_esquerda = centro_x - (raio_topo + (raio_base - raio_topo) * (y - y_topo) / altura)$
 - o $x_direita = centro_x + (raio_topo + (raio_base - raio_topo) * (y - y_topo) / altura)$
3. Desenhar linha horizontal de `x_esquerda` a `x_direita` para cada `y`.

4.4. Exemplo de Código para Desenho de Imagem (Bitmap)

Para desenhar uma imagem (ex: um PNG ou BMP) na tela:

4. **Carregar a imagem:** No Bootloader, você deve carregar o arquivo de disco (requer drivers de disco complexos). No Linux, usa-se `fopen` e `fread` (C) ou `sys_open/sys_read` (Assembly) para ler o arquivo para um buffer.
5. **Desenhar:** Iterar sobre os pixels do buffer da imagem e chamar `draw_pixel` para cada um.

```
; Função para desenhar uma imagem a partir de um buffer
; void draw_image(uint32_t* img_data, int img_w, int img_h, int x_pos, int
y_pos, uint32_t* screen_buffer, int screen_w)

draw_image:
    ; RDI = img_data, RSI = img_w, RDX = img_h, RCX = x_pos, R8 = y_pos,
    R9 = screen_buffer, [RSP] = screen_w

    ; Loop de linhas (y de 0 a img_h)
    ; Loop de colunas (x de 0 a img_w)
    ; Calcular posição na tela: (y_pos + y) * screen_w + (x_pos + x)
    ; Copiar pixel de img_data para screen_buffer
    ret
```